AI Full-Stack Developer

Roadmap with Mega Projects & Practice Problems

This roadmap is structured around 3 evolving Mega Projects that grow with you across every section. Each sub-topic includes 2 targeted mini problems to reinforce your learning before moving forward.

The 3 Mega Projects — Overview

These three projects are chosen to cover the three biggest freelance niches: healthcare, finance, and education. You will start each project as a simple Python script in Section 1 and evolve it all the way to a deployed, monetizable web app by Section 7.

MEGA PROJECT 1 · MediAssist — AI Medical Assistant

Healthcare · High demand · Great for Fiverr Medical Chatbot gigs

Project Statement

Build a command-line AI assistant that accepts patient symptoms as input and returns possible conditions, home remedies, and urgency level (see a doctor / go to ER / safe at home). The assistant must maintain a simple conversation history so follow-up questions are context-aware. By the end of the roadmap this becomes a full web app with user accounts, PDF report export, and a RAG system trained on medical literature.

Section 1 Scope (Python CLI — Week 1–2)

- Accept symptom input from the terminal
- Return 3 possible conditions with a short description
- Classify urgency: Safe / See a Doctor / Emergency
- Keep a list of all symptoms entered in the session
- Save session summary to a .txt file when user types 'exit'

MEGA PROJECT 2 · SpendSense — Smart Expense Tracker

Finance · Universal need · Perfect SaaS product for Section 7

Project Statement

Build an expense tracking system that lets users log daily expenses with category, amount, and notes. The app generates monthly summaries and flags overspending patterns. By the final section it becomes a web app with a dashboard, AI-powered budget advice, and multi-user support — a product you can sell as a SaaS or deliver as a client project on Upwork.

Section 1 Scope (Python CLI — Week 1–2)

- Add an expense: amount, category (food/transport/bills/other), note
- List all expenses for the current session
- Calculate total spending and spending per category
- Find the highest single expense
- Save all data to expenses.csv and reload it on next run

MEGA PROJECT 3 · QuizMind — AI Study Buddy

Education · Fast-growing market · Scalable into a real EdTech product

Project Statement

Build a study assistant that reads a topic from the user and generates quiz questions to test their knowledge. The system tracks scores per session and adapts difficulty based on performance. By the end of the roadmap it becomes a RAG-powered app that can ingest any PDF textbook, generate flashcards, and quiz students — a highly sellable product for students, tutors, and coaching centers.

Section 1 Scope (Python CLI — Week 1–2)

- Accept a topic from the user (e.g. 'Python loops', 'World War 2')
- Generate 5 multiple-choice questions on that topic (hardcoded for now)
- Accept answers and score them
- Show a result summary: X out of 5 correct
- Save score history to quiz_scores.txt with date and topic

How Each Project Evolves — Section by Section

Section	Medical Assistant	Expense Tracker	Study Buddy
S1: Foundation	CLI symptom checker, save .txt report	CLI logger, save to CSV	CLI quiz, save scores to .txt
S2: AI Core	Connect OpenAI — real AI diagnosis responses	AI suggests budget tips from spending data	AI generates quiz questions dynamically
S3: Backend	Flask API — /diagnose endpoint	Django REST API — /add /summary endpoints	Flask API — /quiz /score endpoints
S4: Frontend	HTML form: enter symptoms, see results	HTML dashboard: add expense, view chart	HTML quiz UI with score display
S5: Adv. AI	RAG on medical PDFs — accurate sourced answers	RAG on financial guides — smart advice	Ingest any PDF textbook, auto-quiz
S6: Deploy	Docker + deploy to Render	Docker + deploy to Railway	Docker + deploy to Render
S7: Monetize	Sell as medical chatbot on Fiverr	Sell as SaaS or Upwork client project	Sell to tutors / coaching centres

SECTION 1
FOUNDATION · Week 1 – 2

1.1 Python Fundamentals

Learn variables, data types, conditionals, loops, functions, lists, dicts, and file I/O. This is the absolute foundation — every project you build will use these concepts daily.

Mini Problems — Python Fundamentals	
P1	BMI Calculator: Ask the user for weight (kg) and height (cm). Calculate BMI, print the result and the category (Underweight / Normal / Overweight / Obese). Use functions, conditionals, and formatted output.
P2	Expense Adder (no file yet): Ask the user to enter expenses in a loop (name + amount). When they type 'done', print the total, the most expensive item, and the average spend per item.

1.2 Git & GitHub

Learn init, add, commit, push, pull, branching, and writing a proper README.md. You must push all three Mega Projects to GitHub from Day 1 — treat every commit message as professional practice.

Mini Problems — Git & GitHub	
P1	Repo Setup Challenge: Create a new GitHub repo called 'ai-backend-journey'. Add a professional README with your name, roadmap goals, and a table of contents. Make at least 3 commits showing progress — not one big dump.
P2	Branch & Merge: Create a branch called 'feature/cli-menu' in your MediAssist repo. Add a simple menu (1. Enter symptoms 2. View history 3. Exit) in a new file. Open a Pull Request and merge it into main — write a proper PR description.

1.3 REST APIs (Understanding, not building yet)

Learn what REST APIs are, what HTTP methods (GET, POST, PUT, DELETE) do, and how to call public APIs using Python's 'requests' library. You are consuming APIs here — building comes in Section 3.

🔧 Mini Problems — REST APIs	
P1	Weather Fetcher: Use the Open-Meteo free API (no key needed) to fetch current temperature for Multan. Print: city name, temperature in Celsius, and wind speed. Handle the case where the API is down with a try/except block.
P2	Joke-of-the-Day Bot: Call the JokeAPI (https://v2.jokeapi.dev) to fetch a programming joke. If the joke has a setup/delivery format, print them on separate lines. If it is a single joke, print it directly. Log each joke with timestamp to jokes.txt.

1.4 Small Python Scripts

Practice writing clean, modular scripts. Use functions, separate concerns (input / logic / output), and write at least a short docstring for each function. These scripts are your warm-up reps.

🔧 Mini Problems — Small Python Scripts	
P1	Password Generator: Write a script that generates a strong random password. Accept length and complexity (letters only / letters+numbers / full) as input. Print the generated password and copy it to the clipboard using the 'pyperclip' library.
P2	File Organiser: Given a folder path, scan all files and move them into sub-folders by extension (e.g. all .pdf files go into a 'PDFs' folder, .jpg into 'Images'). Print a summary: how many files were moved into each category.

1.5 Mega Projects — Section 1 Deliverables

MediAssist v0.1 — CLI symptom checker with urgency classification. Saves session report to a .txt file. Pushed to GitHub with a proper README.

SpendSense v0.1 — CLI expense logger with CSV persistence. Shows totals and per-category breakdown. Pushed to GitHub with a proper README.

QuizMind v0.1 — CLI quiz engine on a hardcoded topic. Scores the user and saves results to quiz_scores.txt. Pushed to GitHub with a proper README.

SECTION 2
AI CORE · Week 2 – 4

2.1 OpenAI / Gemini APIs

Set up API keys, understand rate limits, and learn how to send a prompt and receive a completion. Practice with both text and chat (messages array) endpoints.

Mini Problems — OpenAI / Gemini APIs	
P1	First AI Call: Write a Python script that takes a topic from the user and asks OpenAI to explain it in simple terms. Print the response. Add a retry mechanism — if the API call fails, wait 2 seconds and try again (max 3 attempts).
P2	Token Counter: Before sending any prompt, estimate the token count using the 'tiktoken' library. If the prompt exceeds 300 tokens, truncate it and warn the user. Log every API call (prompt, tokens used, response time) to api_log.txt.

2.2 Prompt Engineering

Learn system prompts, few-shot examples, temperature settings, and how prompt structure changes output quality. A well-crafted prompt is the difference between a toy and a product.

Mini Problems — Prompt Engineering	
P1	Tone Switcher: Write a function that takes a user message and a tone parameter ('formal', 'casual', 'technical'). Craft a system prompt for each tone and return the AI response. Show the same message sent in all three tones side-by-side.
P2	Few-Shot Classifier: Build a prompt that classifies a sentence as Positive / Negative / Neutral using 3 examples in the prompt itself (few-shot). Test it on 10 different sentences and print results. Compare results with zero-shot (no examples) to see the difference.

2.3 Build CLI Chatbot

Build a looping chatbot that maintains conversation history in a messages list and sends the full history with each API call so the AI has context.

Mini Problems — CLI Chatbot	
P1	Memory Chatbot: Build a chatbot that remembers the last 10 messages. If history exceeds 10 messages, drop the oldest user+assistant pair. Add a '/clear' command to reset history and a '/history' command to print all messages so far.
P2	Persona Chatbot: Create a chatbot with a fixed persona defined in the system prompt (e.g. 'You are a strict senior Python developer who reviews code and gives blunt feedback'). Test it by pasting bad code into it and see if it stays in character across 5+ turns.

2.4 Text Generator

Build a script that generates structured text output: blog posts, product descriptions, or study notes. Learn to control format using prompt instructions.

Mini Problems — Text Generator	
P1	Blog Post Generator: Accept a topic and target audience from the user. Generate a 300-word blog post with a title, 3 paragraphs, and a call-to-action. Save to a .txt file named after the topic.
P2	Study Notes Maker: Accept any topic and generate structured study notes: Definition, Key Points (bullet list), Common Mistakes, and One Example. Save to notes.txt. This will later feed directly into QuizMind.

2.5 Mega Projects — Section 2 Upgrades

MediAssist v0.2 — Replace hardcoded conditions with a real OpenAI call. System prompt defines the AI as a medical triage assistant. Conversation history maintained per session.

SpendSense v0.2 — After saving expenses, send the CSV data to OpenAI and ask for personalised budget advice. Print the AI tip at the end of each session.

QuizMind v0.2 — Replace hardcoded questions with dynamically generated ones from OpenAI. User picks topic, AI generates 5 MCQs. Score and save as before.

SECTION 3
BACKEND · Week 4 – 6

3.1 Flask / Django

Learn routing, request/response cycle, templates, and how to structure a web project. Use Flask for speed in early projects; Django when you need built-in auth and admin panels.

🔧 Mini Problems — Flask / Django	
P1	Flask Hello API: Create a Flask app with 3 routes: GET /health (returns {status: ok}), GET /about (returns your name and project name), POST /echo (accepts JSON body and returns it back). Test all 3 using Postman or curl.
P2	Django Models: Start a Django project called 'trackr'. Create a model called Expense with fields: amount, category, note, created_at. Run migrations. Use Django shell to add 5 expenses manually and query them by category.

3.2 REST API Development

Build CRUD endpoints (Create, Read, Update, Delete), handle errors with proper HTTP status codes, and validate incoming data. Every professional API follows these patterns.

🔧 Mini Problems — REST API Development	
P1	CRUD Expenses API: Build a Flask REST API with 4 endpoints for expenses: POST /expenses, GET /expenses, GET /expenses/<id>, DELETE /expenses/<id>. Store data in a JSON file (no DB yet). Return proper 201/200/404 status codes.
P2	Input Validation: Add validation to your POST /expenses endpoint. Amount must be a positive number, category must be one of a fixed list, note must be under 100 characters. Return a 400 error with a clear message for any invalid input.

3.3 Database Integration

Connect your API to SQLite (dev) and learn basic ORM patterns with SQLAlchemy or Django ORM. Your data must now survive server restarts — no more JSON files.

Mini Problems — Database Integration	
P1	SQLite Migration: Replace the JSON file storage in your expenses API with an SQLite database using SQLAlchemy. All 4 CRUD endpoints must still work. Add a GET /expenses/summary endpoint that returns total and per-category spend from the DB.
P2	Search & Filter: Add query parameters to GET /expenses: ?category=food to filter by category, ?min=100&max=500 to filter by amount range, ?sort=amount to sort results. All filters must be combinable in one request.

3.4 AI Web App

Combine your Flask backend with an AI API call. Build a single endpoint that accepts user input and returns an AI-generated response — this is the core pattern of every AI product.

Mini Problems — AI Web App	
P1	AI Endpoint: Add a POST /diagnose endpoint to MediAssist's Flask app. It accepts {symptoms: string} and returns {conditions: [], urgency: string, advice: string} from the OpenAI API. Test with Postman. Add a 10-second timeout.
P2	Streaming Response: Upgrade your /diagnose endpoint to stream the AI response token-by-token using Server-Sent Events (SSE). The client should see words appear progressively instead of waiting for the full response.

3.5 Mega Projects — Section 3 Upgrades

MediAssist v0.3 — Full Flask REST API with /diagnose endpoint. SQLite stores session history. Returns structured JSON: conditions, urgency, advice.

SpendSense v0.3 — Django REST API with full CRUD for expenses. SQLite database. /summary and /advice endpoints. Input validation on all fields.

QuizMind v0.3 — Flask API with /quiz (generate questions) and /submit (score answers) endpoints. Quiz sessions stored in SQLite.

SECTION 4
FRONTEND · Parallel Track

4.1 HTML (Forms, Inputs)

Learn semantic HTML, forms, input types, labels, and how browsers submit data. You are building the interface that talks to your backend API.

Mini Problems — HTML Forms & Inputs	
P1	Symptom Form: Build a static HTML page with a textarea for symptoms, a dropdown for age group, and a submit button. Style with basic inline CSS — no frameworks. The form does not need to work yet, just look correct and validate required fields.
P2	Expense Form: Build an HTML form with fields for amount (number), category (select), and note (text). Add HTML5 validation (required, min value). On submit, prevent the default action with JavaScript and console.log the form values as an object.

4.2 CSS Basics

Learn box model, flexbox, colors, typography, and responsive basics. Your UI does not need to be beautiful yet — it needs to be clean, readable, and not embarrassing to show a client.

Mini Problems — CSS Basics	
P1	Card Layout: Style your symptom form inside a centered card: white background, subtle shadow, rounded corners, max-width 500px, vertically centered on the page. The submit button should be full-width with a hover effect. Pure CSS — no frameworks.
P2	Responsive Nav: Build a navigation bar with your project name on the left and 3 links on the right. On mobile (under 600px), the links should stack vertically. Use flexbox and a CSS media query only — no JavaScript.

4.3 JavaScript (Fetch API)

Learn how to call your backend API from the browser using fetch(). Handle loading states, display responses, and manage errors without reloading the page.

🔧 Mini Problems — JavaScript Fetch API	
P1	Live Diagnosis: Connect your symptom HTML form to MediAssist's /diagnose Flask endpoint using fetch(). Show a loading spinner while waiting. Display the conditions and urgency level in a results div below the form. Handle API errors with a user-friendly message.
P2	Expense Dashboard: Build a page that on load calls GET /expenses and renders all expenses in an HTML table. Add a form that calls POST /expenses without reloading the page and instantly appends the new row to the table. Add a delete button per row that calls DELETE /expenses/<id>.

4.4 Mega Projects — Section 4 Upgrades

MediAssist v0.4 — Full HTML/CSS/JS frontend. Symptom form calls /diagnose. Results show conditions, urgency badge (color-coded), and advice. Mobile-friendly.

SpendSense v0.4 — Expense dashboard with add/delete from browser. Live table updates without page reload. Category totals shown as a simple bar using CSS widths.

QuizMind v0.4 — Quiz UI in HTML. User picks topic, clicks Generate, answers MCQs one by one, sees score at the end. All driven by Fetch API calls.

SECTION 5
ADVANCED AI · Week 6 – 8

5.1 RAG — Retrieval-Augmented Generation

RAG lets your AI answer questions based on your own documents, not just its training data. This is the core technology behind 'Chat with PDF' products that clients pay good money for.

Mini Problems — RAG	
P1	Chunk & Retrieve: Take any 10-page PDF, split it into 500-character chunks with 50-character overlap, and store chunks in a Python list. Given a user question, find the top 3 most relevant chunks using simple keyword matching (no vector DB yet). Print the chunks and the question together.
P2	RAG Pipeline: Replace keyword matching with embedding similarity. Embed all chunks and the user query using OpenAI embeddings. Find the top 3 chunks using cosine similarity. Pass chunks + question to OpenAI and return a grounded answer. Track which chunks were used.

5.2 Vector DB (FAISS)

FAISS lets you store millions of embeddings and search them in milliseconds. This replaces your in-memory list and makes your RAG system production-grade.

Mini Problems — Vector DB — FAISS	
P1	FAISS Index: Take 50 sentences from any Wikipedia article. Embed all of them and store in a FAISS index. Save the index to disk. On next run, load the index and search for the top 3 most similar sentences to a new query without re-embedding.
P2	Medical KB: Build a FAISS index from a set of 20 medical FAQs (write them yourself or paste from a reliable source). Query it with a symptom description and return the top 2 matching FAQs. Integrate this retrieval step into MediAssist before the OpenAI call.

5.3 LangChain Basics

LangChain provides ready-made components for chains, memory, and document loaders. Learn the patterns so you can build complex AI pipelines without reinventing the wheel.

Mini Problems — LangChain	
P1	LangChain RAG Chain: Rebuild your RAG pipeline using LangChain's RetrievalQA chain. Use a FAISS vectorstore and the OpenAI LLM. The chain should accept a

	question and return a sourced answer. Compare output quality to your hand-built version from 5.1.
P2	Conversation Memory: Add ConversationBufferWindowMemory (last 5 turns) to your MediAssist chatbot using LangChain. The AI should remember earlier symptoms mentioned in the same session. Test by mentioning a symptom, asking something else, then referring back to the first symptom.

5.4 Chat with PDF App

This is the most popular AI freelance service. Build a complete app where a user uploads a PDF and can ask questions about it. This is a real, sellable product.

🔧 Mini Problems — Chat with PDF	
P1	PDF Loader: Accept a PDF file path, extract all text using PyMuPDF, clean it (remove headers, page numbers, extra whitespace), chunk it, embed it, and store in FAISS. Print: number of pages, number of chunks, and average chunk length.
P2	Full Q&A App: Build a Flask endpoint POST /upload that accepts a PDF and indexes it. Build GET /ask?q=... that runs RAG on the indexed PDF. Build a minimal HTML page to upload the PDF and ask questions. This becomes QuizMind's core feature.

5.5 Mega Projects — Section 5 Upgrades

MediAssist v0.5 — RAG on medical PDFs (e.g. WHO guidelines). AI answers are now grounded in real documents. FAISS index built from medical literature. LangChain memory for multi-turn sessions.

SpendSense v0.5 — RAG on personal finance guides. AI advice is grounded in budgeting best practices, not hallucinated. LangChain chain handles the retrieval + advice pipeline.

QuizMind v0.5 — Full Chat-with-PDF pipeline. Upload any textbook PDF, auto-generate questions from its content. Adaptive difficulty: if user scores below 60%, next round picks harder chunks.

SECTION 6
DEPLOYMENT · Week 7 – 8

6.1 Docker Basics

Docker packages your app and all its dependencies into a container so it runs the same everywhere. Learn Dockerfile, docker build, docker run, and docker-compose for multi-service apps.

🔧 Mini Problems — Docker Basics	
P1	Dockerise Flask: Write a Dockerfile for your MediAssist Flask app. Use python:3.11-slim as base. Copy requirements.txt, install deps, copy app code, expose port 5000, set the CMD. Build the image and run it locally. The /diagnose endpoint must work inside Docker.
P2	Docker Compose: Write a docker-compose.yml that runs two services: your Flask app and a PostgreSQL database. Use environment variables for DB credentials. The Flask app should connect to PostgreSQL (replace SQLite). Add a volumes section so DB data persists.

6.2 Deploy to Render / Railway

Get your app live on the internet with a public URL. Learn how to connect your GitHub repo for auto-deploy, set environment variables in the dashboard, and monitor logs.

🔧 Mini Problems — Deploy	
P1	Live MediAssist: Deploy your Dockerised MediAssist to Render. Set OPENAI_API_KEY as an environment variable in the Render dashboard (never hardcode keys). Share the live URL and confirm the /diagnose endpoint works from Postman using the public URL.
P2	Auto-Deploy Pipeline: Set up GitHub Actions to run your tests (write at least 2 pytest tests for your API) on every push to main. If tests pass, Render auto-deploys. If tests fail, the deploy is blocked. Commit a failing test, push, and confirm the deploy was blocked.

6.3 Environment Variables

Never hardcode API keys or secrets. Learn .env files, python-dotenv, and how to manage different configs for dev vs production. This is a professional non-negotiable.

🔧 Mini Problems — Environment Variables	
P1	.env Audit: Go through all three of your Mega Projects and remove every hardcoded secret (API keys, DB passwords). Replace with os.getenv() calls. Create a .env.example file listing all required variables without their values. Add .env to .gitignore.
P2	Config Class: Create a config.py file in one of your projects that defines a Config base class and two subclasses: DevelopmentConfig (SQLite, debug=True) and ProductionConfig (PostgreSQL, debug=False). Load the correct class based on a FLASK_ENV environment variable.

6.4 Mega Projects — Section 6 Upgrades

MediAssist v0.6 — Fully Dockerised. Deployed to Render with public URL. PostgreSQL in production. Auto-deploy on push to main. Zero hardcoded secrets.

SpendSense v0.6 — Dockerised with docker-compose (app + DB). Deployed to Railway. All secrets in environment variables. Basic pytest suite with auto-deploy gate.

QuizMind v0.6 — Deployed to Render. PDF uploads stored in a temp folder with cleanup after session. Public URL works for PDF upload and quiz flow end-to-end.

SECTION 7
MONETIZATION

7.1 Fiverr / Upwork Profile

Set up your profile with your three Mega Projects as portfolio pieces. Your Fiverr gig title, description, and screenshots must be professional. Your GitHub must look active.

Mini Problems — Fiverr / Upwork	
P1	Gig Package: Write three Fiverr gig packages (Basic / Standard / Premium) for MediAssist as a product: title, description, delivery time, price, and what is included at each tier. Use your deployed live URL as the demo link.
P2	Portfolio README: Upgrade the README of each Mega Project on GitHub to be client-facing: a GIF demo (record with Loom), a Features list, a Live Demo link, and a 'Hire Me' section with your Fiverr/Upwork link.

7.2 AI Chatbots for Business

Learn how to package your AI skills as business solutions — customer support bots, FAQ bots, and internal knowledge base bots. These are the most common client requests.

Mini Problems — AI Chatbots for Business	
P1	FAQ Bot: Build a chatbot that answers questions about a fictional restaurant (menu, hours, location, booking). Use RAG on a restaurant info document you write yourself. Embed in a simple HTML chat UI. This is a real deliverable you can sell.
P2	Lead Capture Bot: Extend your chatbot to collect name, email, and query from the user during the conversation. At the end of the session, save the lead to a CSV. This is a feature every local business client will pay for.

7.3 Automation Tools

Automation is a massive freelance market. Learn to combine Python, APIs, and AI to build time-saving tools that clients will pay recurring fees for.

Mini Problems — Automation Tools	
P1	Email Summariser: Use the Gmail API (or any SMTP-accessible inbox) to fetch the last 10 emails. Send each subject + body to OpenAI and return a one-line summary. Output: a daily digest of 10 emails in under 100 words total. Schedule it to run every morning using a cron job.
P2	Report Generator: Extend SpendSense to auto-generate a monthly PDF report with: total spend, per-category breakdown, top 3 expenses, and an AI budget recommendation. Use ReportLab or WeasyPrint. Email the PDF to the user automatically on the 1st of each month.

7.4 Mega Projects — Final State

MediAssist v1.0 — Listed as a Fiverr gig. Deployed, RAG-powered, multi-turn memory, PDF report export. Client-facing README and live demo.

SpendSense v1.0 — Monthly auto-PDF report. Listed on Upwork as a SaaS product. Lead capture optional. Recurring revenue potential.

QuizMind v1.0 — Sold to tutors and coaching centers. Upload any textbook, auto-quiz students. Score tracking. Can be white-labelled for clients.

SECTION 8
SCALING

8.1 Advanced Backend

Add authentication (JWT), rate limiting, background tasks (Celery), and caching (Redis). These are the patterns that separate hobby projects from professional products.

Mini Problems — Advanced Backend	
P1	JWT Auth: Add user registration and login to SpendSense using JWT tokens. Every expense endpoint must require a valid token. Each user sees only their own expenses — multi-tenancy enforced at the DB query level.
P2	Background Tasks: Add a Celery task to MediAssist that processes long AI responses asynchronously. The POST /diagnose endpoint returns immediately with a task_id. A GET /result/<task_id> endpoint returns the result when ready. Use Redis as the message broker.

8.2 System Design

Learn how to think about scalability: load balancing, horizontal scaling, caching strategies, and database indexing. Even as a freelancer, understanding system design makes you more valuable.

Mini Problems — System Design	
P1	Design Doc: Write a 1-page system design document for MediAssist at scale (10,000 concurrent users). Cover: load balancer, multiple app servers, database (read replicas), FAISS index sharing strategy, and CDN for static assets. Diagrams allowed — use draw.io.
P2	DB Indexing: Add database indexes to SpendSense on the most common query columns (user_id, category, created_at). Use EXPLAIN QUERY PLAN in SQLite to compare query speed before and after. Write a short report of what you found.

8.3 Internship / Jobs

Your three Mega Projects are now your portfolio. Prepare your CV, GitHub, and LinkedIn. Target AI backend roles at startups — they value shipped products over degrees.

🔧 Mini Problems — Internship / Jobs	
P1	CV Project Section: Write the Projects section of your CV for all three Mega Projects. Use the STAR format: what it does, what tech stack, what problem it solves, and one measurable outcome (e.g. 'Reduces diagnosis time by presenting top 3 conditions in under 2 seconds').
P2	Cold Outreach: Write a cold LinkedIn message to 3 AI startup founders whose companies are building products similar to your projects. Reference their product specifically, mention your relevant project, and offer to show a live demo. Draft all 3 messages — different tone each time.

What You Will Have Built

By the end of this roadmap you will have three production-grade, deployed, AI-powered web applications — each with its own GitHub repo, live URL, and Fiverr/Upwork listing. You will have solved 48 targeted practice problems across all sub-topics, giving you the muscle memory to write backend AI code confidently and independently.

MediAssist AI medical triage assistant · RAG on medical PDFs · Multi-turn memory · Deployed · Listed on Fiverr

SpendSense Smart expense tracker · AI budget advice · Auto PDF report · Multi-user JWT auth · Listed on Upwork

QuizMind PDF-to-quiz engine · Adaptive difficulty · Tutor-ready · White-label capable · Deployed on Render

Start today. Commit daily. Ship everything.